

JAVA SWİNG İLE MODÜLER KARGO YÖNETİM SİSTEMİ TASARIMI

Nesne Yönelimli Analiz (OOA) ve Görsel Yazılım Mimarisi
Raporu

Hüsna Nur Ulu

04/01/2026 (Versiyon:1.0)

İÇİNDEKİLER

1. Giriş

1.1. Problem Tanımı

1.2. Problem Amacı

2. Önerilen Sistem Tasarımı

2.1. Gereksinimler ve Kısıtlar

2.2. Use-Case Diyagramı

2.3. Sequence Diyagramları

2.4. Sınıf Diyagramı

3. Sonuç

1. Giriş

Bu çalışma, lojistik sektöründeki karmaşık iş akışlarını dijitalleştirmek amacıyla geliştirilmiş bir Proof of Concept (Kavram Kanıtı) projesidir.

1.1. Problem Tanımı

Bir kargo firması, her gün binlerce paketi doğru alıcılara ulaştırmak zorundadır. Böylesine yüksek işlem sirkülasyonunun manuel tablolar, defterler veya ilkel yöntemlerle yönetilmesi operasyonel açıdan imkansız hale gelmektedir. Manuel sistemlerde yaşanan veri giriş hataları, paketlerin kaybolması ve kurye-paket eşleşmesindeki aksaklıklar; hem firma çalışanlarının iş yükünü artırmakta hem de müşteri hizmetlerinde büyük bir yoğunluğa sebep olmaktadır. Müşterilerin kargo durumlarını anlık olarak takip edememesi, kurumsal verimliliği düşürmekte ve hizmet kalitesini zayıflatmaktadır. Bu bağlamda, süreçlerin dijital bir otomasyon sistemine aktarılması kaçınılmaz bir ihtiyaç olarak ortaya çıkmaktadır.

1.2. Problem Amacı

Bu projenin temel hedefi, 1.1. bölümünde detaylandırılan operasyonel aksaklıkları ve manuel yönetimden kaynaklanan sorunları kökten çözmek adına Java tabanlı, nesne yönelimli bir kargo otomasyon sistemi geliştirmektir. Proje kapsamında; hataları ve aksamaları minimize eden, veriye anlık erişim imkanı tanıyan ve kullanıcı dostu bir arayüz ile desteklenen dijital bir altyapı oluşturulması amaçlanmaktadır. Bu sistem sayesinde kargo süreçlerinin her aşaması şeffaf ve izlenebilir hale getirilerek, hem çalışanların iş yükünün hafifletilmesi hem de müşteri hizmetlerindeki yoğunluğun optimize edilmesi hedeflenmektedir. Nihayetinde, nesne tabanlı programlama prensipleriyle inşa edilmiş, sürdürülebilir ve güvenilir bir yazılım çözümü elde edilmiş olacaktır.

2. Önerilen Sistem Tasarımı

2.1 Gereksinimler ve Kısıtlar

2.1.1. Fonksiyonel Gereksinimler (Sistemin yapması gereken işler):

Kargo Kaydı: Sistem, yeni bir kargo gönderisi için gönderici adı, alıcı adı, kargonun hacmine göre araç tipi belirleyip kargoyu bir kuryeye atmalıdır.

Takip Numarası Üretme: Sistem, her yeni kargo kaydı için benzersiz (unique) rastgele bir takip numarası oluşturmaktadır.

Durum Güncelleme: Yetkili kurye, kargonun durumunu "Kabul Edildi", "Yolda", "Dağıtımda" veya "Teslim Edildi" olarak güncelleyebilmelidir.

Kurye Yönetimi: Sistem, aktif kuryelerin isim ve ID bilgilerini saklayabilmeli ve her kargoyu bir kurye ile ilişkilendirebilmelidir.

Kargo Sorgulama: Kullanıcı, takip numarası ile kargonun mevcut durumunu ve hangi kuryede olduğunu görüntüleyebilmelidir.

2.1.2. Fonksiyonel Olmayan Gereksinimler (Sistemin kalite özellikleri):

Kullanılabilirlik: Sistem, teknik bilgisi az olan personelin bile rahatça kullanabileceği basit bir Görsel Arayüz (GUI) sunmalıdır.

Hata Yönetimi: Geçersiz veri girişlerinde sistem kullanıcıyı uyarmalı ve hatalı kaydı engellemelidir.

Performans: Kayıt ve sorgulama işlemleri makul bir sürede (anlık olarak) gerçekleşmelidir.

2.1.3. Kısıtlar:

Programlama Dili: Proje Java dili kullanılarak geliştirilmelidir.

IDE: Geliştirme sürecinde NetBeans kullanılacaktır.

Arayüz: Proje, komut satırı üzerinden değil, Java Swing kütüphanesi ile hazırlanmış görsel bir pencere üzerinden çalışmalıdır.

Veri Saklama: Veriler çalışma zamanında (Runtime) nesneler üzerinde tutulacak veya basit dosya sistemleri (txt/dat) üzerinden yönetilecektir.

2.2. Use-Case Diyagramı

Sistemimizde üç temel aktör bulunmaktadır: **Personel**, **Kurye** ve **Müşteri**.

Sistem Aktörleri ve Görevleri:

1.Yönetici (Admin/Şube Çalışanı):

-Kargo Kaydı Oluştur ve Kuryeye Ata

-Kurye Tanımla

2.Kurye:

-Kargo Durumu Güncelle (Yolda/Teslim Edildi vb.)

3.Müşteri:

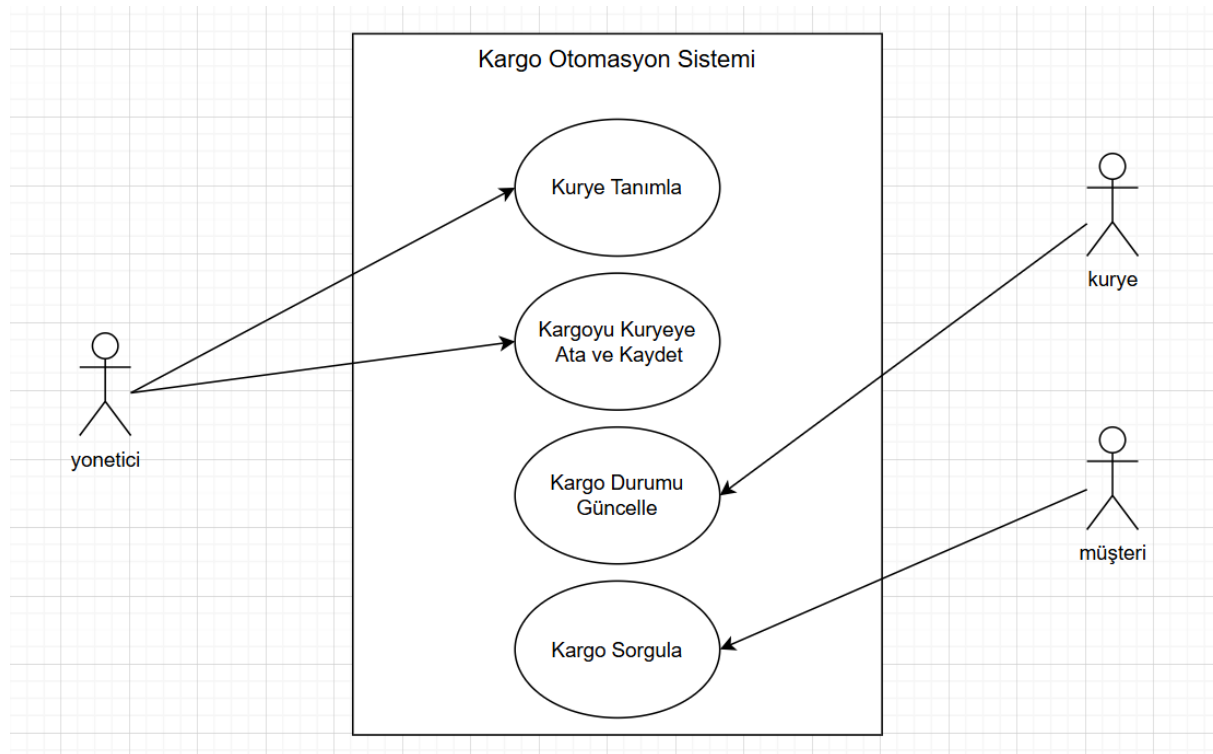
-Kargo Sorgula

Bu Diyagramın Açıklaması:

Yönetici, sistemin yönetici kısmını temsil eder. Kargoları ve kuryeleri sisteme dahil eden ana aktördür.

Kurye, operasyonel süreci yönetir. Kargo eline geçtiğinde veya teslim ettiğinde durumu günceller.

Müşteri, sisteme veri girişi yapamaz; sadece kargonun nerede olduğunu izleyebilir.



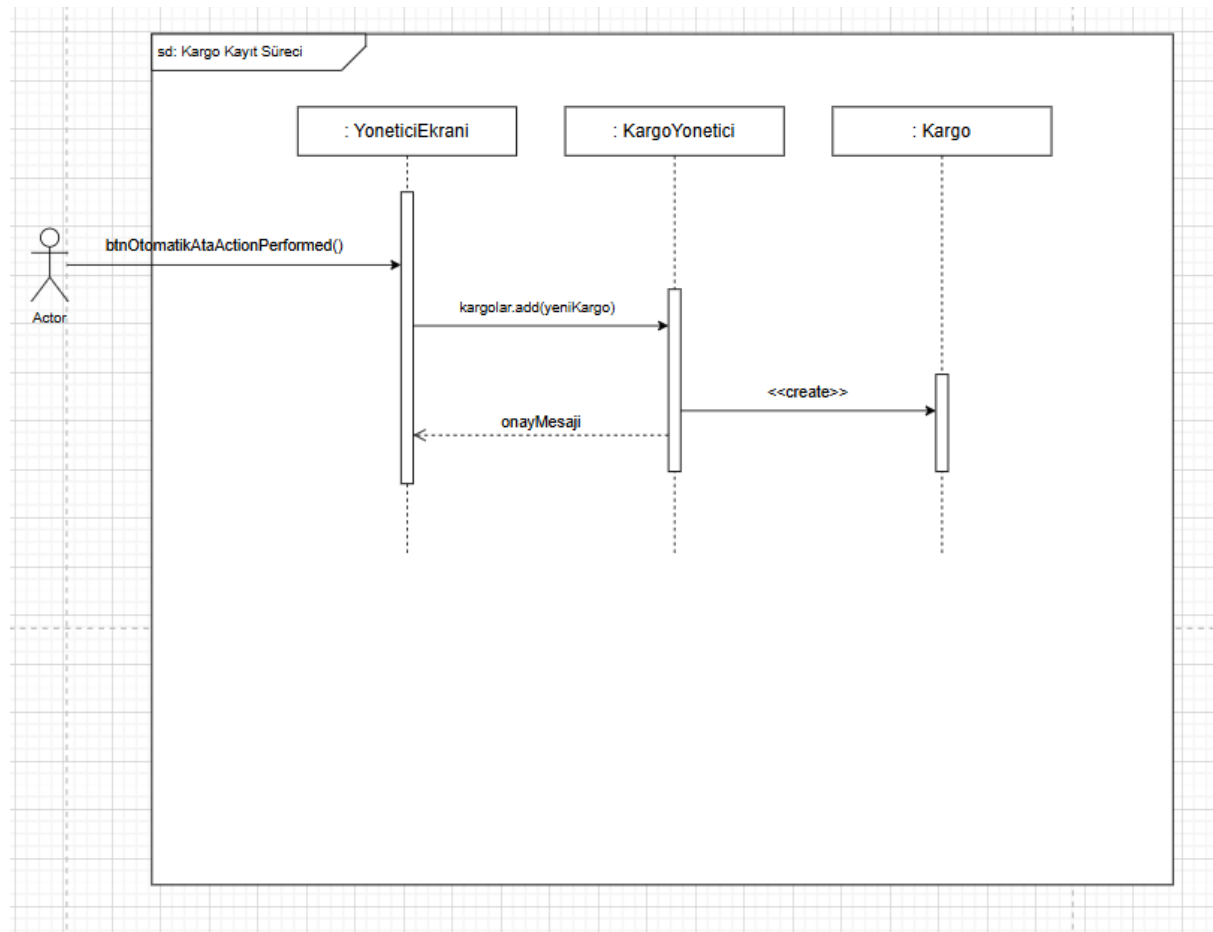
Sistem Etkileşim Modeli (Use-Case): Yönetici, Kurye ve Müşteri rollerinin sistem üzerindeki yetki sınırlarını ve iş akışlarını tanımlayan kullanım durumu diyagramı.

2.3 Sequence Diyagramları

Bu bölümde, sistemin temel işleyişini oluşturan senaryolarda nesnelerin birbirleriyle olan etkileşimi, metod çağrıları ve mesajlaşma trafiği modellenmiştir. Sistem; yönetici, müşteri ve kurye olmak üzere üç ana akış üzerinden kurgulanmıştır.

2.3.1. Kargo Kayıt Süreci

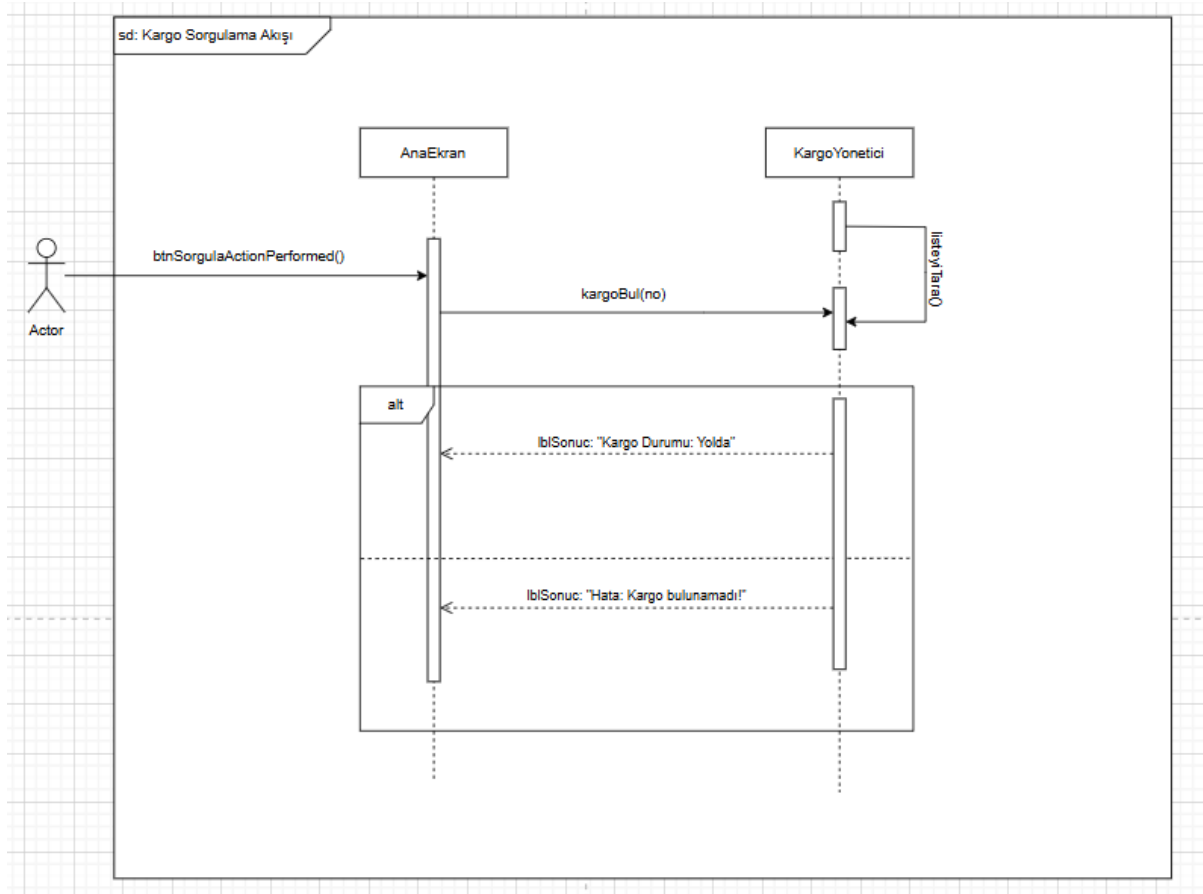
Kargo kayıt süreci, yöneticinin arayüz üzerinden gönderici, alıcı ve araç tipi verilerini girmesiyle başlar. **YoneticiEkranı**, topladığı bu verileri **KargoYonetici** kontrolcü sınıfına iletir. Sistem, çalışma zamanında (runtime) yeni bir **Kargo** nesnesi örneği (instance) oluşturur ve kullanıcıya işlemin başarı durumu hakkında geri bildirim sağlar.



(Şema 1)

2.3.2. Kargo Sorgulama Süreci

Müşterinin takip numarası ile yaptığı sorgulama işlemidir. **KargoYonetici** sınıfı, veri listesini tarayarak ilgili numaraya ait eşleşmeyi bulur. Bu süreçte kullanılan "**alt**" (**alternatif**) yapısı sayesinde; kargo bulunursa kargonun güncel durumu ve **kurye bilgisi** arayüze döndürülür, bulunamaması durumunda ise kullanıcıya hata mesajı iletir.



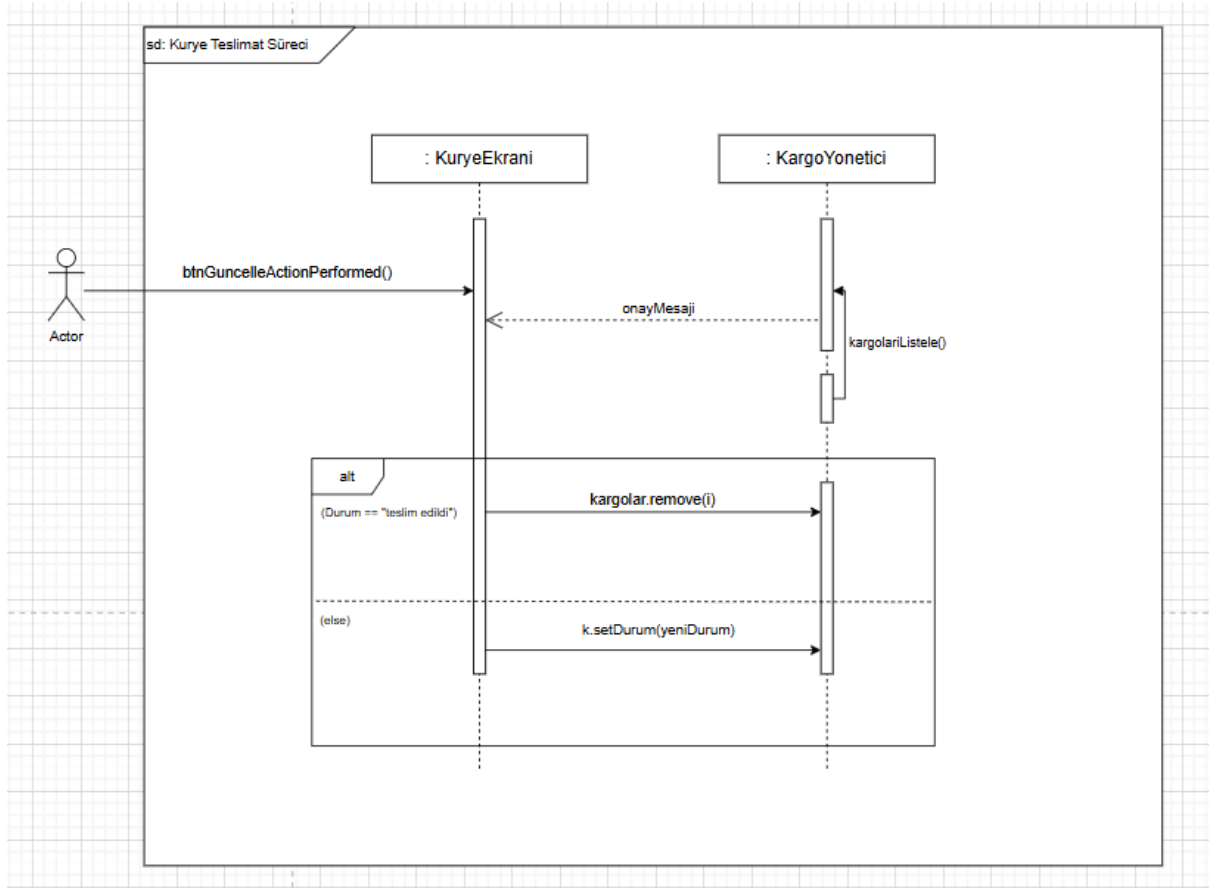
(Şema 2)

Şema (1 ve 2): Nesne Etkileşim Akışı (Sequence Diagram): Kargo kayıt ve kurye atama süreçlerinde nesneler arası mesaj iletimini ve operasyonel zaman akışını gösteren teknik şema.

2.3.3.Kurye Teslimat Süreci

Kuryenin kendisine atanmış kargoların durumunu değiştirdiği süreçtir. Kurye, **KuryeEkranı** üzerinden bir durum seçtiğinde sistem **"alt"** yapısını kullanarak kontrol sağlar:

- **Seçilen durum "Teslim Edildi" ise:** İlgili kargo nesnesi veri bütünlüğünü korumak adına merkezi listeden silinir.
- **Diğer durumlar:** Kargonun durum değişkeni güncellenir. İşlem sonunda arayüzdeki **self-call (kargolariListele)** metodu ile kurye tablosu anlık olarak güncellenir.



Yazılım Mimarisi (Class Diagram): Kalıtım (Inheritance), Soyutlama (Abstraction) ve İlişkilendirme (Association) prensipleri üzerine kurulu modüler sınıf yapısı.

2.4. Sınıf Diyagramı

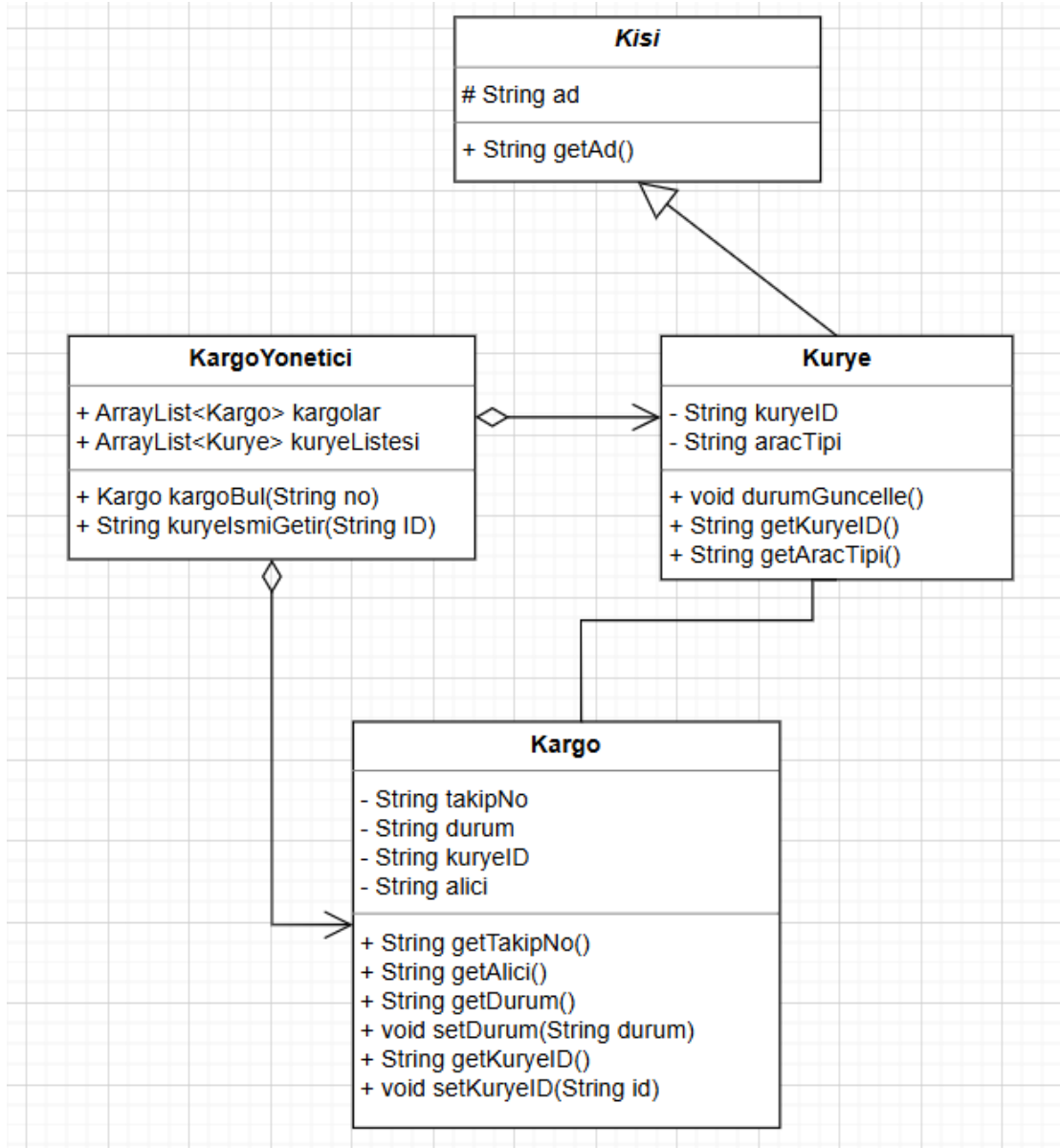
Sistemin statik yapısını temsil eden sınıf diyagramı, Nesne Yönelimli Programlama (OOP) prensiplerine uygun olarak tasarlanmıştır. Temel yapı şu bileşenlerden oluşmaktadır:

Kalıtım (Inheritance): Kisi sınıfı abstract (soyut) olarak tanımlanmış olup; ad gibi ortak özellikleri barındırır. Kurye sınıfı bu sınıftan türetilerek kod tekrarı önlenmiştir.

Merkezi Yönetim (Controller): KargoYonetici sınıfı, sistemdeki tüm kargo ve kurye verilerini ArrayList yapılarında tutan merkezi bir kontrolör görevi görür.

İlişkisel Yapı: Kargo nesnesi, içerisinde doğrudan bir kurye nesnesi yerine kuryeID tutarak veri tabanı mantığıyla hafifletilmiş, sorgulama anında bu ID üzerinden KargoYonetici yardımıyla kurye ismine dinamik olarak erişim sağlanmıştır.

Erişim Belirleyiciler: Veri gizliliği (Encapsulation) ilkesine göre değişkenler private (-) ve protected (#) olarak tutulmuş, verilere erişim public (+) getter/setter metotları ile sınırlandırılmıştır.



3. Sonuç

3.1. Sistemin Problemi Çözme Yaklaşımı

Bu otomasyon, lojistik süreçlerdeki manuel takip hatalarını merkezi bir kontrol yapısıyla dijitalleştirerek çözmektedir. Sistem; kargo kayıt, otomatik kurye atama ve anlık durum takibi özellikleriyle karmaşık iş akışlarını tek bir platformda birleştirir. Böylece veri kaybı önlenirken, yöneticiden müşteriye kadar tüm paydaşlar için şeffaf bir bilgi akışı sağlanmış olur.

3.2. Tasarım Kararları ve Gerekçeleri

Yazılım mimarisinde verilerin tek bir merkezden yönetilmesi kararı, tüm ekranlarda veri tutarlılığını korumak amacıyla alınmıştır. Dinamik giriş kontrolü sayesinde sisteme yeni

eklenen personelin anında işlem yapabilmesi sağlanırken, bellek performansını yüksek tutmak için teslim edilen kargoların otomatik silinmesi kararlaştırılmıştır. Bu tercihler, uygulamanın hem hızlı hem de güvenilir çalışmasını sağlamıştır.

3.3. Gelecek Çalışmalar ve Geliştirme Önerileri

Sistem, ilerleyen aşamalarda verilerin kalıcı saklanması için bir veritabanı entegrasyonu ile güçlendirilebilir. Ayrıca kuryeler için harita tabanlı rota optimizasyonu, müşteriler için kişiselleştirilmiş profil sayfaları ve online ödeme modülleri eklenerek projenin ticari kapasitesi artırılabilir. Bu geliştirmeler, mevcut stabil altyapıyı tam kapsamlı bir lojistik platformuna dönüştürecektir.